國立政治大學商學院資訊管理學系
碩士學位論文
Department of Management Information Systems
College of Commerce
National Chengchi University (NCCU)
Master Thesis

基於抽象精煉的遞迴神經網路強健性驗證
Robustness Verification of Recurrent Neural Networks with Abstraction Refinement

指導教授：洪智鐸 教授
Advisor: Chih-Duo Hong
研究生: 林立仁
Li-Jen Lin

中華民國 115 年 5 月
May, 2026

摘要

遞迴神經網絡（RNNs）的認證式局部強健性驗證是一個困難的問題, 因為非線性近似所引入的近似誤差會透過遞迴連結傳遞，並隨時間累積。因此，可擴展的線性邊界近似往往過於保守，無法驗證成功實際上安全的輸入，尤其當許多預激活區間帶有較大的近似誤差時更為明顯。本研究提出一套適用於 RNN 驗證的抽象精煉框架，透過切割這類區間，在每個分支上消除主要的近似誤差。對於 RELU，於零點切割可使分支變為精確；對於 TANH、SIGMOID 等平滑激活函數(同時也出現在 LSTM 的閘門中)，我們選擇能最小化過度近似面積的切點 p^*，並以離線預先計算、驗證時以查表方式取得。為控制長序列中切割的組合成本，我們提出以 SHAP 為導向的選擇策略，依各隱藏層神經元對驗證目標的貢獻排序，再依時序對最關鍵者進行精煉。在 CIFAR10、MNIST stroke 與 MNIST 上、橫跨 vanilla RNN 與 LSTM 的實驗顯示，本方法相較於僅用抽象的基準，在驗證成功率與強健性邊界緊度上都有一致提升，同時呈現出 RELU 與平滑激活函數模型之間明確的執行時間取捨。

關鍵詞: 政治大學, 強健性驗證, 形式化方法, 遞迴神經網路, 人工智慧安全, 模型檢驗.

Abstract

Certified robustness verification for recurrent neural networks (RNNs) is hard because relaxation errors from nonlinear activations propagate through the recurrence and accumulate, leaving linear bound propagation too conservative to certify many robust inputs. We propose an abstraction-refinement framework that splits over-approximated pre-activation intervals to cut the dominant relaxation error per branch: at zero for RELU, making it exact, and at an offline-computed point p^* for TANH and SIGMOID, retrieved by table lookup. A SHAP-guided strategy keeps splitting tractable by refining only the most critical neurons in temporal order. Across CIFAR10, MNIST stroke, and MNIST on RNNs and LSTMs, our method consistently improves verification success and bound tightness over abstraction-only baselines.

Keywords: NCCU, Robustness Verification, Formal Methods, Recurrent Neural Networks, Artificial Intelligence Safety, Model Checking.

Contents

List of Figures

List of Tables

1 Introduction

Recurrent Neural Networks (RNNs) have proven to be highly effective for tasks that involve sequential data, with successful applications spanning natural language processing (Sutskever et al., 2014), speech recognition (Graves et al., 2013), financial time series forecasting (Nelson et al., 2017), and trajectory prediction for autonomous vehicles (Alahi et al., 2016). Despite these successes, RNNs share a known weakness with other neural network architectures: they are susceptible to adversarial perturbations. These are small, deliberately constructed changes to the input that mislead the model while being nearly invisible to a human observer (Goodfellow et al., 2015, Szegedy et al., 2013).

This vulnerability poses serious risks in high-stakes applications. For instance, an RNN used to predict vehicle trajectories in an autonomous driving system could be manipulated to produce dangerous outputs (Li et al., 2021). In medical settings, RNNs that monitor patient vital signs over time (Che et al., 2018) could be fooled into producing incorrect diagnoses or treatment recommendations. Similarly, financial systems that rely on RNNs for market analysis are at risk of adversarial interference leading to significant monetary harm (Cherny et al., 2018).

Many defenses against adversarial attacks have been studied (Cohen et al., 2019, Madry et al., 2017), but most offer no formal guarantees and can be bypassed by stronger attacks. Certified robustness verification takes a different approach: it provides a mathematical proof that for a given clean input sequence $\mathbf{X}_0$ with label j, the model's top-1 prediction will not change for any perturbed input $\mathbf{X}$ that stays within a specified ℓ_p ball of radius ε at each timestep.

Even with recent advances, obtaining tight certificates for RNNs is still a hard problem. The most practical scalable verifiers use abstraction-based bound propagation, which

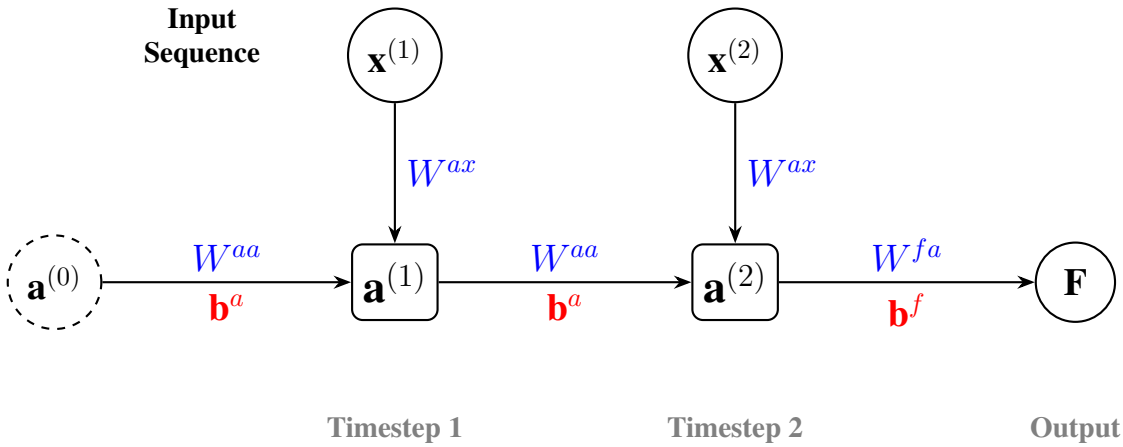
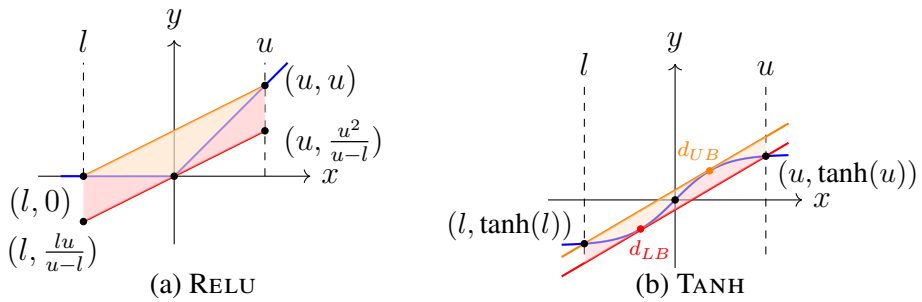


Figure 1.1: A vanilla RNN architecture with 2 timesteps.



(a) RELU (b) TANH

Figure 1.2: Linear relaxation for RELU and TANH when the pre-activation interval $[l, u]$ straddles zero. The shaded regions represent the over-approximation introduced by the relaxation.

approximates nonlinear activation functions with linear bounds in order to derive certified output ranges (Ko et al., 2019, Li et al., 2023). The core difficulty occurs when a neuron's pre-activation value can be either positive or negative—that is, when the interval $[l, u]$ straddles zero. In this situation, the gap introduced by the linear approximation can be large for both piecewise-linear activations like RELU and smooth activations like TANH and SIGMOID, as illustrated in Figure 1.2. Because RNNs feed the hidden state back into themselves at every step, this approximation error accumulates across timesteps, leading to overly conservative bounds and frequent false negatives: inputs that are actually robust but cannot be certified.

Two characteristics of RNNs make robustness verification more difficult for RNNs than for standard feedforward networks. First, the hidden state at each timestep encodes information from all prior steps, so an imprecise bound computed early in the sequence will carry its error through every recurrent update that follows. Second, if the recurrent weight

matrix has a large spectral norm, even a small approximation error can grow substantially over the course of a long sequence, making it progressively harder to maintain tight bounds.

Abstraction refinement offers a principled way to address this conservatism. When the initial abstract bounds are not tight enough to certify robustness, the verifier can split unstable pre-activation intervals into sub-intervals and solve the resulting smaller sub-problems, each with tighter activation approximations. In the RNN setting, however, this is more involved than it is for feedforward networks. Splitting a neuron at timestep t constrains the feasible set of hidden states from that point onward, affecting every subsequent recurrent step. If refinement is applied blindly to all cross-zero neurons, the number of sub-problems grows exponentially and the procedure quickly becomes intractable.

This thesis proposes a one-neuron-at-a-time refinement workflow designed specifically for RNN verification. When bound computation alone fails to certify a sample, we identify a small number of high-impact neurons and split them one at a time. For a chosen neuron (t, r) with pre-activation interval $[l_r^{(t)}, u_r^{(t)}]$, we select a split point p^* and create two branches that restrict the interval to $[l_r^{(t)}, p^*]$ and $[p^*, u_r^{(t)}]$, respectively. We then recompute the abstract bounds through all remaining timesteps and attempt to certify each branch independently. For RELU, we split at $p^* = 0$, which exactly eliminates the cross-zero approximation error on the selected neuron. For TANH, the split point need not be zero; each branch then operates over a narrower interval that yields a substantially tighter linear envelope.

For vanilla RNNs with RELU activation, splitting at zero is the optimal choice because zero is the exact kink point of the activation. For smooth activations such as TANH and SIGMOID—which appear in both vanilla RNNs and LSTM gates—splitting at zero is not necessarily the best option. The ideal split point depends on how curved the activation function is over the specific interval $[l, u]$, not on the sign of the pre-activation value. To handle this, we extend our framework to LSTMs and introduce a lookup-table-based strategy for selecting the split point: for each interval $[l, u]$, we precompute offline the point $p^* \in (l, u)$ that minimizes the total linear relaxation error over the two resulting sub-intervals $[l, p^*]$ and $[p^*, u]$. At verification time, p^* is retrieved by a nearest-neighbor lookup into this table. Using tighter linear envelopes on each branch leads directly to

tighter certified output bounds and higher verification rates.

To determine which neurons to split, we use gradient-based SHAP attributions computed on the hidden states to rank each neuron by how much its approximation error contributes to the verification failure. From this ranked list, we select candidate neurons whose pre-activation intervals still incur a non-negligible relaxation gap, sort them in temporal order, and apply splits within a fixed depth budget. Processing earlier timesteps first is a deliberate choice: a split at an early step tightens the bounds at all subsequent recurrent transitions, so the benefit compounds over the rest of the sequence. This strategy allows us to concentrate the refinement budget on the most impactful neurons without exhaustively branching over all candidates.

We evaluate our approach on CIFAR10 (reshaped as pixel sequences) and MNIST stroke trajectories using vanilla RNN classifiers with RELU and TANH activations, and further extend the evaluation to LSTM classifiers on MNIST to assess the effectiveness of p^*-based splitting on gated architectures. Across model sizes, sequence lengths, and perturbation radii, SHAP-guided single-neuron refinement recovers samples that abstraction alone fails to certify and consistently tightens certified robustness margins, with runtime overhead that scales predictably with sequence length and split depth. For LSTMs, p^*-based splitting yields meaningful verification gains at shorter sequence lengths, though the benefit diminishes as sequence length grows due to compounded relaxation errors across the four gate computations per timestep.

This thesis makes three main contributions. First, we develop a splitting-based abstraction refinement workflow for RNN verification, where each split is incorporated into the backward recursion so that it tightens the bounds at all subsequent timesteps. Second, we introduce a neuron ranking strategy based on gradient-based SHAP attributions, which prioritizes neurons whose relaxation error has the greatest impact on the verification outcome while respecting the temporal structure of the recurrence. Third, we present an empirical evaluation on two sequence classification benchmarks, showing that our method improves certification rates and produces tighter output bounds compared to abstraction alone, along with an analysis of the runtime trade-offs across different activation functions and sequence lengths.

The remainder of this thesis is organized as follows. Chapter 2 surveys related work on RNN robustness verification, abstract interpretation, and abstraction refinement strategies. Chapter 3 introduces the abstraction-based bound propagation framework that serves as the foundation for our approach. Chapter 4 presents the split framework, covering both the splitting rule and the SHAP-guided neuron selection procedure. Chapter 5 reports the results of our experimental evaluation. Chapter 6 concludes the thesis and outlines directions for future work.

2 Related Work

Certified robustness verification for neural networks has developed along two broad lines: abstraction-based methods that propagate certified bounds through network layers, and refinement-based methods that tighten those bounds when the initial abstraction is too conservative. This chapter reviews both lines of work, organized into (i) robustness verification for neural networks with an emphasis on RNN-specific challenges, (ii) abstract interpretation and interval propagation as the foundational bounding machinery, and (iii) abstraction refinement and branching strategies. We highlight where existing approaches leave a gap that our framework addresses: targeted, temporally-ordered refinement guided by attribution scores on hidden states.

2.1 Neural Network Robustness Verification

Adversarial attacks (Madry et al., 2017, Szegedy et al., 2013) demonstrated that neural networks can be misled by imperceptible input perturbations, motivating certified robustness verification as a formal alternative to empirical testing. For feed-forward networks (FNNs), scalable verifiers based on abstract interpretation produce certified bounds using intervals (Gehr et al., 2018, Zhang et al., 2018), zonotopes (Singh et al., 2018), and polyhedral domains (Singh et al., 2019). These methods trade completeness for efficiency by replacing nonlinear activations with linear relaxations. Complete solvers such as RELUPLEX and MARABOU (Katz et al., 2017, 2019) avoid this approximation but face scalability limits on deep or high-dimensional models.

RNN verification inherits these foundations but is further complicated by *recurrent unrolling*: approximation errors introduced at one timestep propagate through recurrent

weights and compound over subsequent timesteps. POPQORN (Ko et al., 2019), a representative linear bound propagation-based verifier for RNNs, extends linear bound propagation to RNNs and serves as the abstraction baseline that our framework refines. Other approaches develop RNN-specific bounds via interval certification (Du et al., 2021, Zhang et al., 2021, 2023), differential verification (Mohammadinejad et al., 2021), and reachability-based methods such as star sets (Tran et al., 2023). While these methods advance the underlying abstractions, none incorporate *targeted refinement* to address the dominant failure mode we study: overly loose relaxations when many pre-activations straddle zero, followed by temporal amplification of that conservatism across the unrolled recurrence.

2.2 Abstract Interpretation

Abstract interpretation (Cousot and Cousot, 1977) provides the theoretical basis for sound over-approximation in program analysis and, more recently, in neural network verification. Applied to neural networks, the idea is to propagate an abstract element—an interval, zonotope, or polyhedron—through each layer in place of concrete values, obtaining certified output bounds without enumerating all possible inputs. Interval-based methods (Gehr et al., 2018, Zhang et al., 2018) are the most lightweight but can accumulate significant over-approximation across layers. Zonotope-based methods (Singh et al., 2018) capture linear correlations between neurons and yield tighter bounds at moderate cost. Polyhedral domains (Singh et al., 2019) achieve the tightest abstractions among these three but incur higher computational overhead.

For RNNs, the key challenge is that the abstract element must be propagated not only through feedforward layers but also through recurrent connections over multiple timesteps. POPQORN (Ko et al., 2019) addresses this by deriving affine upper and lower envelopes for nonlinear activations over their pre-activation intervals and recursively composing these envelopes backward through the unrolled recurrence. Our framework adopts this two-phase abstraction-a forward pass that bounds each pre-activation interval, followed by the backward recursion described above that composes the activation envelope into output bounds-as its baseline, and invokes refinement only when the resulting bounds are insuffi-

cient to certify robustness, preserving the efficiency of bound computation in the common case.

2.3 Abstract Refinement

When an initial abstraction fails to certify robustness, refinement tightens bounds by conditioning on additional constraints over activation regions. For FNNs, branch-and-bound frameworks split activation domains at zero for piecewise-linear activations (Bunel et al., 2020, De Palma et al., 2021, Wang et al., 2021), yielding exact behavior on each branch. Other methods strengthen relaxations via cutting planes and hybrid convex constraints (Zhou et al., 2024). A particularly relevant thread is debugging-oriented refinement (Singh et al., 2018), which identifies high-impact regions of over-approximation and strategically splits zero-crossing pre-activation intervals to eliminate the loosest relaxation regimes.

Refinement for sequential and stateful networks has begun to emerge. Polyhedral RNN verification can incorporate gradient-driven refinement (Ryou et al., 2021), and recent branch-and-bound schemes handle general nonlinearities with branching-point optimization (Shi et al., 2025). Reachability-based analysis for stateful deep systems can also introduce adaptive refinement (Du et al., 2020). However, these methods treat all nodes in the unrolled computation graph uniformly, without accounting for how a split at an early timestep propagates through recurrent connections to tighten bounds at later timesteps—the core temporal structure that distinguishes RNN refinement from its FNN counterpart.

Our framework addresses this gap directly. We integrate attribution-based prioritization into the refinement loop: gradient-based SHAP attributions (Lundberg and Lee, 2017, Sundararajan et al., 2017) on hidden states rank each cross-zero neuron by its contribution to the verification objective, and refinement is applied in temporal order so that early splits propagate their tightening effect through the remaining recurrence. This design yields a targeted search strategy that respects the temporal leverage inherent to RNN unrolling, rather than treating timesteps as interchangeable layers.

3 Robustness Verification via Abstraction

3.1 Problem Formulation

Consider an RNN classifier $F : \mathbb{R}^{n \times m} \to \mathbb{R}^c$, where n is the per-timestep input dimension, m is the sequence length, and c is the number of classes. For an input sequence $\mathbf{X}_0 = [\mathbf{x}_0^{(1)}, \ldots, \mathbf{x}_0^{(m)}]$ with ground-truth label j, our goal is to verify that the top-1 classification remains unchanged under input perturbations. The adversarially perturbed sequence is denoted as $\mathbf{X} = [\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}]$, where each perturbed frame $\mathbf{x}^{(k)}$ is constrained within an ε-radius ℓ_p-ball: $\mathbf{x}^{(k)} \in \mathbb{B}_p(\mathbf{x}_0^{(k)}, \varepsilon)$. Specifically, we want to certify that the ground-truth logit remains dominant for all such perturbed inputs; a sufficient condition is

$$\gamma_j^L > \gamma_i^U, \quad \forall\, i \neq j,$$

where γ_j^L and γ_i^U are certified scalar lower and upper bounds on the respective logits over the entire perturbation set.

The challenge lies in computing bounds that are tight enough to make this verification feasible. Loose bounds lead to false negatives: inputs that are actually robust but cannot be certified because the computed interval $[\gamma_j^L, \gamma_j^U]$ overlaps with a competing class. We address this through a two-phase procedure. First, we apply bound propagation to establish initial certified bounds, following the abstraction-first paradigm of POPQORN (Ko et al., 2019). We refer to the forward computation of hidden pre-activation bounds as forward bound computation (FBC) and to the subsequent computation of certified output bounds as

backward bound recursion (BBR). When initial bounds fail to certify robustness, Chapter 4 introduces the abstraction-refinement procedure that tightens them.

3.2 Abstraction-Based Bound Computation

3.2.1 Bounding Nonlinear Activations

The verification procedure derives input-dependent linear bounds $F_j^L(\mathbf{X})$ and $F_j^U(\mathbf{X})$ for each logit F_j, then optimizes them over the perturbation set to obtain scalar certificates γ_j^L and γ_j^U. Concretely, for any perturbed sequence $\mathbf{X}$ with $\mathbf{x}^{(k)} \in \mathbb{B}_p(\mathbf{x}_0^{(k)}, \varepsilon)$, BBR constructs $F_j^L(\mathbf{X})$ and $F_j^U(\mathbf{X})$ such that

$$F_j^L(\mathbf{X}) \leq F_j(\mathbf{X}) \leq F_j^U(\mathbf{X}).$$

The key operation in BBR is approximating each nonlinear activation $\sigma(\cdot)$ with a pair of affine bounds. FBC first computes sound pre-activation bounds $\mathbf{l}^{(t)}, \mathbf{u}^{(t)} \in \mathbb{R}^s$ satisfying $l_r^{(t)} \leq y_r^{(t)} \leq u_r^{(t)}$ for each neuron r at timestep t. Given these bounds, BBR uses affine lower and upper envelopes on σ:

$$h_{L,r}^{(t)}(v) = \alpha_{L,r}^{(t)} v + \beta_{L,r}^{(t)}, \quad h_{U,r}^{(t)}(v) = \alpha_{U,r}^{(t)} v + \beta_{U,r}^{(t)}, \tag{3.1}$$

such that $h_{L,r}^{(t)}(v) \leq \sigma(v) \leq h_{U,r}^{(t)}(v)$ for all $v \in [l_r^{(t)}, u_r^{(t)}]$.

We index a hidden neuron by (t, r), meaning neuron r at timestep t, and call it cross-zero when $l_r^{(t)} < 0 < u_r^{(t)}$. The envelope coefficients depend on which case applies.

For RELU, the non-cross-zero cases are exact: if $l_r^{(t)} \geq 0$, the activation is the identity, so $(\alpha_{L,r}^{(t)}, \beta_{L,r}^{(t)}) = (\alpha_{U,r}^{(t)}, \beta_{U,r}^{(t)}) = (1, 0)$; if $u_r^{(t)} \leq 0$, the activation is constantly zero, so both pairs equal $(0, 0)$. When $l_r^{(t)} < 0 < u_r^{(t)}$, exact representation is no longer possible and the tightest valid linear envelope is:

$$\alpha_{U,r}^{(t)} = \alpha_{L,r}^{(t)} = \frac{u_r^{(t)}}{u_r^{(t)} - l_r^{(t)}}, \quad \beta_{L,r}^{(t)} = 0, \quad \beta_{U,r}^{(t)} = -\frac{l_r^{(t)} u_r^{(t)}}{u_r^{(t)} - l_r^{(t)}}.$$

For TANH and SIGMOID, which share the same S-shape (convex for $v < 0$, concave for $v > 0$, with an inflection at the origin), the envelope depends on where $[l_r^{(t)}, u_r^{(t)}]$ lies relative to zero. We write σ for either activation and drop the (t) and r indices in the cases below. When the pre-activation bounds fall within the non-negative interval, $l \geq 0$, σ is concave, so the chord lies below the curve and every tangent lies above it. We take the secant through the endpoints as the lower envelope,

$$\alpha_L = \frac{\sigma(u) - \sigma(l)}{u - l}, \qquad \beta_L = \sigma(l) - \alpha_L \, l,$$

and the tangent at the midpoint $\mu = \frac{l+u}{2}$ as the upper envelope,

$$\alpha_U = \sigma'(\mu), \qquad \beta_U = \sigma(\mu) - \alpha_U \, \mu.$$

When $u \leq 0$, σ is convex and the two roles swap, with the secant becoming the upper envelope and the midpoint tangent the lower envelope.

When the interval is cross-zero, it spans the inflection point, so neither the chord nor a single tangent bounds σ on its own. The upper envelope is the line through the left endpoint $(l, \sigma(l))$ that is tangent to the curve at a point $d_U \in (0, u]$ satisfying

$$\sigma'(d_U) = \frac{\sigma(d_U) - \sigma(l)}{d_U - l};$$

symmetrically, the lower envelope is the line through the right endpoint $(u, \sigma(u))$ that is tangent at a point $d_L \in [l, 0)$ satisfying

$$\sigma'(d_L) = \frac{\sigma(d_L) - \sigma(u)}{d_L - u}.$$

We take the midpoint as the tangent point in the two monotone-curvature cases because, on a convex or concave segment, the tangent at any interior point is a valid envelope, and the midpoint is a symmetric choice that keeps the larger endpoint gap small without solving an optimization. The width of the remaining gap is governed by how much σ curves over $[l, u]$, precisely the quantity our refinement targets, since splitting the interval shortens

each sub-interval and reduces its curvature-induced gap (Chapter 4).

3.2.2 Derivation for a 2-Timestep RNN

We show the full derivation for the 2-timestep many-to-one RNN in Figure 1.1, which captures the essential backward recursion. The general m-timestep case follows by applying the same construction iteratively.

The network equations are:

$$\begin{aligned} F(\mathbf{X}) &= \mathbf{W}^{fa}\mathbf{a}^{(2)} + \mathbf{b}^{f}, \\ \mathbf{a}^{(2)} &= \sigma(\mathbf{y}^{(2)}), \quad \mathbf{y}^{(2)} = \mathbf{W}^{aa}\mathbf{a}^{(1)} + \mathbf{W}^{ax}\mathbf{x}^{(2)} + \mathbf{b}^{a}, \\ \mathbf{a}^{(1)} &= \sigma(\mathbf{y}^{(1)}), \quad \mathbf{y}^{(1)} = \mathbf{W}^{aa}\mathbf{a}^{(0)} + \mathbf{W}^{ax}\mathbf{x}^{(1)} + \mathbf{b}^{a}, \end{aligned}$$

where $\mathbf{a}^{(0)} \in \mathbb{R}^s$ is the initial hidden state and $\mathbf{y}^{(t)} \in \mathbb{R}^s$ is the pre-activation vector at timestep t.

Upper Bound Derivation

Starting from the output layer and letting $\mathbf{c}_{j,:}^{(2)} \coloneqq \mathbf{W}_{j,:}^{fa}$ denote the initial backward coefficient row, each coordinate selects the upper or lower activation envelope according to the sign of $c_{j,r}^{(t)}$ to preserve a valid upper bound:

$$\lambda_{j,r}^{(t)} = \begin{cases} \alpha_{U,r}^{(t)}, & c_{j,r}^{(t)} \geq 0, \\ \alpha_{L,r}^{(t)}, & c_{j,r}^{(t)} < 0; \end{cases} \qquad \delta_{j,r}^{(t)} = \begin{cases} \beta_{U,r}^{(t)}, & c_{j,r}^{(t)} \geq 0, \\ \beta_{L,r}^{(t)}, & c_{j,r}^{(t)} < 0. \end{cases}$$

Define $\Lambda_{j,:}^{(t)} \coloneqq \mathbf{c}_{j,:}^{(t)} \odot \lambda_{j,:}^{(t)}$ and $\eta_j^{(t)} \coloneqq \langle \mathbf{c}_{j,:}^{(t)}, \delta_{j,:}^{(t)} \rangle$. Here $\Lambda_{j,:}^{(t)}$ aggregates the per-neuron slopes into the effective backward coefficient row, while $\eta_j^{(t)}$ collects the per-neuron intercepts through an inner product over the hidden dimension, which is valid because the envelopes are written in the form $\alpha v + \beta$ and the intercept of $c_{j,r}^{(t)} h_r$ contributes $c_{j,r}^{(t)}\beta_r$ to the constant term. Applying Equation (3.1) at timestep 2 and then setting $\mathbf{c}_{j,:}^{(1)} \coloneqq \Lambda_{j,:}^{(2)}\mathbf{W}^{aa}$ to recurse

into timestep 1 yields the linear upper bound:

$$F_j(\mathbf{X}) \;\le\; \Lambda_{j,:}^{(1)}\mathbf{W}^{aa}\mathbf{a}^{(0)} + \sum_{z=1}^{2}\Lambda_{j,:}^{(z)}\mathbf{W}^{ax}\mathbf{x}^{(z)} + \sum_{z=1}^{2}\left(\Lambda_{j,:}^{(z)}\mathbf{b}^{a} + \eta_j^{(z)}\right) + b_j^{f}.$$

Since this bound is linear in each perturbed frame, maximizing over $\mathbf{x}^{(z)} \in \mathbb{B}_p(\mathbf{x}_0^{(z)}, \varepsilon)$ via Hölder's inequality (Hardy et al., 1952) yields the certified scalar upper bound:

$$\begin{aligned}\gamma_j^{U} = {} & \Lambda_{j,:}^{(1)}\mathbf{W}^{aa}\mathbf{a}^{(0)} + \sum_{z=1}^{2}\varepsilon\|\Lambda_{j,:}^{(z)}\mathbf{W}^{ax}\|_q \\ & + \sum_{z=1}^{2}\Lambda_{j,:}^{(z)}\mathbf{W}^{ax}\mathbf{x}_0^{(z)} + \sum_{z=1}^{2}\left(\Lambda_{j,:}^{(z)}\mathbf{b}^{a} + \eta_j^{(z)}\right) + b_j^{f},\end{aligned}$$

where q is the dual norm of p satisfying $1/p + 1/q = 1$.

Lower Bound Derivation

The lower bound follows the same backward recursion with inverted envelope selection. BBR maintains a separate coefficient row $\widetilde{\mathbf{c}}_{j,:}^{(t)}$, initialized as $\widetilde{\mathbf{c}}_{j,:}^{(2)} \coloneqq \mathbf{W}_{j,:}^{fa}$ and recursed by $\widetilde{\mathbf{c}}_{j,:}^{(1)} \coloneqq \Omega_{j,:}^{(2)}\mathbf{W}^{aa}$. We keep $\widetilde{\mathbf{c}}_{j,:}^{(t)}$ distinct from $\mathbf{c}_{j,:}^{(t)}$ because, although the two rows coincide at the output layer ($\widetilde{\mathbf{c}}_{j,:}^{(2)} = \mathbf{c}_{j,:}^{(2)} = \mathbf{W}_{j,:}^{fa}$), they diverge after one recursion step, since the lower-bound row propagates through $\Omega_{j,:}^{(2)}$ rather than $\Lambda_{j,:}^{(2)}$. Each coordinate now selects the lower or upper envelope according to the sign of $\widetilde{c}_{j,r}^{(t)}$ to preserve a lower bound:

$$\omega_{j,r}^{(t)} = \begin{cases}\alpha_{L,r}^{(t)}, & \widetilde{c}_{j,r}^{(t)} \ge 0,\\ \alpha_{U,r}^{(t)}, & \widetilde{c}_{j,r}^{(t)} < 0;\end{cases} \qquad \theta_{j,r}^{(t)} = \begin{cases}\beta_{L,r}^{(t)}, & \widetilde{c}_{j,r}^{(t)} \ge 0,\\ \beta_{U,r}^{(t)}, & \widetilde{c}_{j,r}^{(t)} < 0.\end{cases}$$

Define $\Omega_{j,:}^{(t)} \coloneqq \widetilde{\mathbf{c}}_{j,:}^{(t)} \odot \omega_{j,:}^{(t)}$ and $\zeta_j^{(t)} \coloneqq \langle \widetilde{\mathbf{c}}_{j,:}^{(t)}, \theta_{j,:}^{(t)} \rangle$. After the same two-step recursion, the certified scalar lower bound is:

$$\begin{aligned}\gamma_j^{L} = {} & \Omega_{j,:}^{(1)}\mathbf{W}^{aa}\mathbf{a}^{(0)} - \sum_{z=1}^{2}\varepsilon\|\Omega_{j,:}^{(z)}\mathbf{W}^{ax}\|_q \\ & + \sum_{z=1}^{2}\Omega_{j,:}^{(z)}\mathbf{W}^{ax}\mathbf{x}_0^{(z)} + \sum_{z=1}^{2}\left(\Omega_{j,:}^{(z)}\mathbf{b}^{a} + \zeta_j^{(z)}\right) + b_j^{f}.\end{aligned}$$

3.3 Illustrative Example

We use the 2-timestep RNN in Figure 1.1 to walk through a concrete bound computation with all weights equal to one and all biases equal to zero. The input perturbation is $\varepsilon = 1.5$ around a clean input $x = 1$ at each timestep, and the initial hidden state is $\mathbf{a}^{(0)} = 0$.

Forward Computation of Pre-Activation Bounds. At timestep 1, the clean pre-activation is $y^{(1)} = 1\times0+1\times1+0 = 1$. Under the perturbation, $\mathbf{x}^{(1)} \in [-0.5, 2.5]$, which propagates directly to $y^{(1)} \in [-0.5, 2.5]$. Since this interval straddles zero, neuron $(1, 1)$ is cross-zero and requires the approximate RELU envelope.

At timestep 2, the pre-activation $y^{(2)} = \mathbf{W}^{aa}\mathbf{a}^{(1)} + \mathbf{W}^{ax}\mathbf{x}^{(2)} + \mathbf{b}^{a}$ depends on $\mathbf{a}^{(1)} = \sigma(y^{(1)})$, so the uncertainty from timestep 1 compounds with the new input perturbation. Computing the bounds through the recurrent connection yields $y^{(2)} \in [-0.92, 5.0]$, illustrating how approximation errors accumulate over time.

ReLU Linearization. Both timesteps fall into the cross-zero case, and the envelope coefficients are:

Timestep 1:

$$\alpha_{U,1}^{(1)} = \frac{2.5}{2.5-(-0.5)} \approx 0.83, \quad \beta_{U,1}^{(1)} = -\frac{(-0.5)(2.5)}{3.0} \approx 0.42, \quad \alpha_{L,1}^{(1)} = 0.83, \quad \beta_{L,1}^{(1)} = 0.$$

Timestep 2:

$$\alpha_{U,1}^{(2)} = \frac{5.0}{5.0-(-0.92)} \approx 0.84, \quad \beta_{U,1}^{(2)} \approx 0.80, \quad \alpha_{L,1}^{(2)} = 0.84, \quad \beta_{L,1}^{(2)} = 0.$$

Certified Output Bounds. The slopes feed into the backward recursion through the effective weight product $\Lambda_{j,:}^{(z)}\mathbf{W}^{ax}$. Because each slope is strictly less than one, each effective recurrent coefficient is contracted by a factor $\prod_k \alpha_{U,1}^{(k)} < 1$ at each timestep. Completing the backward recursion produces certified bounds of $[-0.77, 5.0]$, a range of width 5.77. This wide interval directly reflects the triangular over-approximation regions accumulated at both timesteps, and it is too loose to certify robustness for most reasonable perturbation

radii. Section 4 revisits this example to show how splitting only one cross-zero neuron narrows the bounds enough to recover the certificate.

3.4 Extension to LSTMs

Long Short-Term Memory networks (LSTMs) (Hochreiter and Schmidhuber, 1997) introduce a cell state $\mathbf{c}^{(t)}$ alongside the hidden state $\mathbf{a}^{(t)}$, with four interacting gates controlling information flow:

$$
\begin{aligned}
\mathbf{i}^{(t)} &= \sigma(\mathbf{W}^{ix}\mathbf{x}^{(t)} + \mathbf{W}^{ia}\mathbf{a}^{(t-1)} + \mathbf{b}^{i}),\\
\mathbf{f}^{(t)} &= \sigma(\mathbf{W}^{fx}\mathbf{x}^{(t)} + \mathbf{W}^{fa}\mathbf{a}^{(t-1)} + \mathbf{b}^{f}),\\
\mathbf{g}^{(t)} &= \textsc{Tanh}(\mathbf{W}^{gx}\mathbf{x}^{(t)} + \mathbf{W}^{ga}\mathbf{a}^{(t-1)} + \mathbf{b}^{g}),\\
\mathbf{o}^{(t)} &= \sigma(\mathbf{W}^{ox}\mathbf{x}^{(t)} + \mathbf{W}^{oa}\mathbf{a}^{(t-1)} + \mathbf{b}^{o}),\\
\mathbf{c}^{(t)} &= \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \mathbf{g}^{(t)},\\
\mathbf{a}^{(t)} &= \mathbf{o}^{(t)} \odot \textsc{Tanh}(\mathbf{c}^{(t)}),
\end{aligned}
$$

where σ denotes the sigmoid activation, $\odot$ denotes the element-wise product, and each gate has separate input-to-gate weights ($\mathbf{W}^{ix}$, $\mathbf{W}^{fx}$, $\mathbf{W}^{gx}$, $\mathbf{W}^{ox}$) and hidden-to-gate weights ($\mathbf{W}^{ia}$, $\mathbf{W}^{fa}$, $\mathbf{W}^{ga}$, $\mathbf{W}^{oa}$).

The verification goal remains the same, namely to certify that the top-1 prediction is stable under ℓ_p perturbations of radius ε at each timestep. The key additional complexity over vanilla RNNs is that bounding the hidden state $\mathbf{a}^{(t)}$ now requires bounding products of gate activations and the cell state, each of which is itself a nonlinear function of the previous hidden state. Following the construction of Section 3.2.2, we handle these gate-cell products by applying a second layer of affine relaxation. Whereas the vanilla RNN relaxes a single one-dimensional activation $\sigma(v)$ per neuron, the LSTM couples two pre-activations through products such as $\sigma(\mathbf{y}^{f(t)}) \odot \mathbf{c}^{(t-1)}$ and $\sigma(\mathbf{y}^{i(t)}) \odot \textsc{Tanh}(\mathbf{y}^{g(t)})$, so the relaxation must bound a two-dimensional nonlinear surface rather than a one-dimensional curve. We bound each such surface by a pair of affine planes $h_L(v, z) = \alpha_L v + \beta_L z + \gamma_L$ and $h_U(v, z) = \alpha_U v + \beta_U z + \gamma_U$, whose coefficients are obtained by solving a constrained

volume-minimization problem over the input ranges (Ko et al., 2019). Given pre-activation bounds for each gate, we first derive these planes for the output-cell product, the forget-cell product, and the input-cell-gate product, then compose them through the same backward recursion used for vanilla RNNs. This produces sound bounds on $\mathbf{c}^{(t)}$ and, after a further TANH relaxation, on $\mathbf{a}^{(t)}$.

The resulting FBC/BBR pipeline for LSTMs follows the same backward recursion structure as for vanilla RNNs, with the difference that each timestep now involves four sets of gate pre-activations rather than one. Unstable pre-activation intervals in any of the four gates introduce over-approximation, and these errors accumulate jointly through both the cell-state and the hidden-state paths. This richer error structure means that refinement for LSTMs must consider gate neurons in addition to hidden-state neurons.

3.5 Limitations and Motivation for Refinement

The FBC/BBR framework provides a sound and efficient procedure for certified robustness verification, but its precision degrades in the presence of many unstable pre-activations. The cross-zero envelope for RELU introduces a triangular over-approximation whose width is proportional to $u_r^{(t)} \cdot |l_r^{(t)}|/(u_r^{(t)} - l_r^{(t)})$; for TANH and SIGMOID, analogous gaps arise from the curvature of the smooth activation over the pre-activation interval. In isolation, each such gap is small, but in an RNN these local errors are fed back through the recurrent weight matrix at every subsequent timestep. When the spectral norm of $\mathbf{W}^{aa}$ is close to or greater than one, the accumulated error can grow rapidly with sequence length, yielding bounds that are far too conservative to certify actually robust inputs.

As illustrated in Section 3.3, even a 2-timestep network with a moderate perturbation radius can produce a certified interval wide enough to cause a false negative. For the longer sequences used in our experiments (m up to 45), this conservatism becomes the dominant failure mode. The refinement framework presented in Chapter 4 addresses this by selectively splitting the most harmful unstable intervals, creating sub-problems in which the dominant source of error is eliminated for RELU and sharply reduced for the smooth activations, while keeping the total number of sub-problems tractable.

4 Split: Abstraction Refinement for RNN Verification

Bound computation can become overly conservative when many pre-activation intervals incur large relaxation gaps, which for RELU occurs when the interval straddles zero and for smooth activations grows with the interval width. In RNNs, these local errors are amplified through the recurrent map across timesteps, yielding false negatives: inputs that are actually robust but cannot be certified. We mitigate this by splitting pre-activation ranges, producing sub-problems with exact RELU behavior and substantially tighter TANH or SIGMOID envelopes. For RELU, splitting at zero is optimal because zero is its kink point, making each resulting branch exact. For smooth activations such as TANH and SIGMOID, which arise in both vanilla RNNs and in the gates of LSTMs, splitting at zero is not guaranteed to yield the tightest envelope. We therefore adopt a more principled strategy that selects a split point p^* minimizing the total linear-approximation area across both branches, at the cost of a one-time pre-computation per model. The derivation and lookup-table construction for p^* are detailed in Section 4.2.

4.1 Verification Workflow

As shown in Figure 4.1, verification begins with the original input $\mathbf{X}_0$ and perturbation radius ε. Certified abstract output bounds $[\gamma^L, \gamma^U]$ are computed by forward bound computation (FBC), which derives pre-activation bounds at each timestep, followed by backward bound recursion (BBR). If the robustness margin holds, that is, the lower bound of the ground-truth class exceeds all competing upper bounds, the procedure terminates

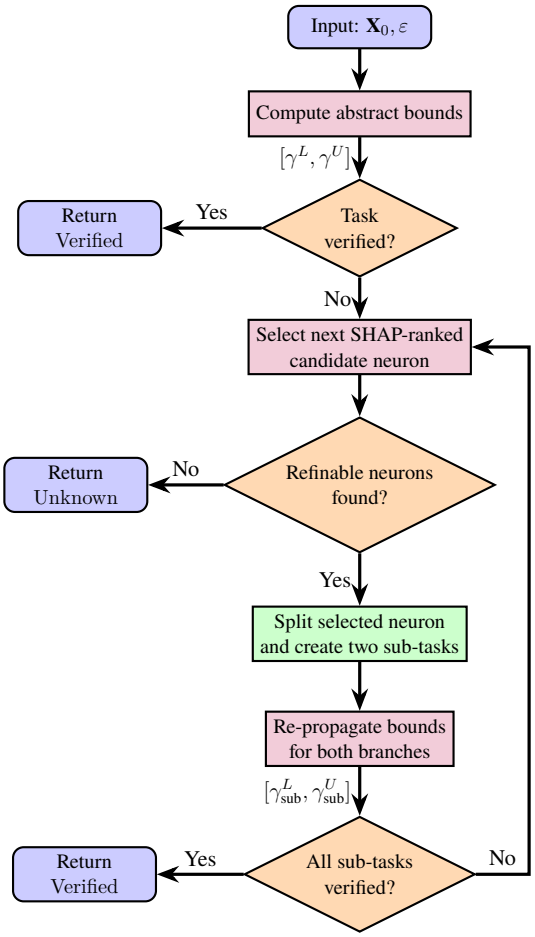
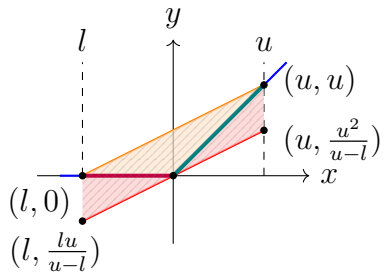


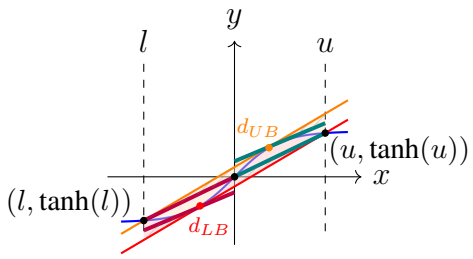
Figure 4.1: Verification workflow for local robustness of RNN.

and returns Verified. Otherwise, we select the next critical neuron from the SHAP-ranked candidate set described in Section 4.3. If the candidate set is exhausted or the depth budget is reached, the result is returned as Unknown. When a critical neuron r^* at timestep t_c is found, we split its pre-activation interval at a chosen point p^* to create positive and negative sub-tasks, constraining that neuron to $[p^*, u_{r^*}^{(t_c)}]$ or $[l_{r^*}^{(t_c)}, p^*]$, respectively, then recompute the output bounds for each sub-task with FBC and BBR, carrying the refined constraint through the remaining unrolled timesteps. Certification is achieved only if all generated sub-tasks satisfy the margin; otherwise, the procedure continues by selecting the next critical neuron and further refining the remaining failing branches. The choices of p^* depend on the activation type and is described in Section 4.2.

Algorithm 1 expresses the workflow of Figure 4.1 in procedural form. Lines 1 to 4 correspond to the initial abstraction and the early-exit check, while lines 5 and 6 correspond to candidate selection and recursive refinement. The following sections detail each



(a) RELU



(b) TANH

Figure 4.2: Splitting the pre-activation interval tightens the relaxation, becoming exact for RELU and substantially tighter for TANH.

component in turn.

Algorithm 1 Overall Refinement Workflow

Input: Clean input sequence $\mathbf{X}$, perturbation radius ε, norm p, ground-truth label j, selection count ρ, maximal depth d_{max}
Output: True iff $\mathbf{X}$ is certified robust under (ε, p)

$[\mathbf{l}, \mathbf{u}] \leftarrow \text{FBC}(\mathbf{X}, \varepsilon, p)$
$[\gamma^L, \gamma^U] \leftarrow \text{BBR}([\mathbf{l}, \mathbf{u}], \mathbf{X}, \varepsilon, p)$
if $\gamma_j^L > \max_{i \neq j} \gamma_i^U$ **then**
    return True
end if
$\mathcal{N}_{sel} \leftarrow \text{SELECT}(\mathbf{X}, \varepsilon, p, j, \rho)$
return $\text{DIVIDE}(\mathbf{X}, \varepsilon, p, j, 0, d_{\text{max}}, \mathcal{N}_{sel}, \text{None})$

4.2 Split Rule

Given a timestep t_c, a selected neuron r^* with pre-activation bounds $[l_{r^*}^{(t_c)}, u_{r^*}^{(t_c)}]$, and a split point $p^* \in (l_{r^*}^{(t_c)}, u_{r^*}^{(t_c)})$, we create two branches by constraining only the selected neuron: the positive branch sets $l_{\text{pos},r^*}^{(t_c)} \leftarrow p^*$, and the negative branch sets $u_{\text{neg},r^*}^{(t_c)} \leftarrow p^*$. Each branch produces its own certified output bounds $[\gamma^L, \gamma^U]$, and the original sample is certified if and only if every branch satisfies

$$\gamma_j^L > \max_{i \neq j} \gamma_i^U, \quad \text{where } j \text{ is the top-1 class label.} \tag{4.1}$$

For RELU, only neurons whose pre-activation interval crosses zero, that is, $l_{r^*}^{(t_c)} < 0 < u_{r^*}^{(t_c)}$, carry nonzero relaxation error; intervals confined to a single sign are already exact. We set $p^* = 0$, the unique optimal split point for RELU: the positive branch reduces

the activation to the identity on $[0,\, u_{r^*}^{(t_c)}]$ and the negative branch to the zero function on $[l_{r^*}^{(t_c)},\, 0]$, eliminating the triangular over-approximation entirely. Splitting a single-sign RELU neuron leaves both branches exact and the bounds unchanged, so in the RELU setting only sign-changing neurons yield improvement.

For smooth activations such as TANH and SIGMOID, every bounded pre-activation interval carries nonzero relaxation error regardless of sign, and the optimal split point is not fixed at zero. The tightest linear envelope depends on the curvature of σ over each subinterval and varies with the pre-activation bounds $[l_{r^*}^{(t_c)},\, u_{r^*}^{(t_c)}]$ computed in each branch. Tighter linear relaxations at the selected neuron generally translate into improved tightness of the certified output bounds. We therefore seek a point that minimizes the relaxation error when the pre-activation interval at the selected neuron is divided into $[l_{r^*}^{(t_c)},\, p]$ and $[p,\, u_{r^*}^{(t_c)}]$. From Equation (3.1), we denote the upper and lower bound lines on $[l_{r^*}^{(t_c)},\, p]$ as $\overline{h}_1 = \overline{\alpha}_1 x + \overline{\beta}_1$ and $\underline{h}_1 = \underline{\alpha}_1 x + \underline{\beta}_1$, and those on $[p,\, u_{r^*}^{(t_c)}]$ as $\overline{h}_2 = \overline{\alpha}_2 x + \overline{\beta}_2$ and $\underline{h}_2 = \underline{\alpha}_2 x + \underline{\beta}_2$.

We quantify tightness by integrating the gap between the lower and upper linear relaxations, where a smaller gap indicates a tighter relaxation. The vertical distance $\overline{h}(v) - \underline{h}(v)$ at a given v is the width of the band of values that the relaxation admits but the true activation does not, so integrating it over the interval measures the total area of the over-approximation region. Because the certified output bound is monotone in these per-neuron over-approximation areas, a relaxation with a smaller integrated gap yields tighter downstream bounds, which makes the integrated area a faithful objective for choosing the split point. We formalize this as a tightness loss $P(\sigma, l, u, p)$ for the activation function $\sigma(x)$ over the input range $[l, u]$ with a branching point p:

$$P(\sigma, l, u, p) = \int_l^p \left[\overline{h}_1(v) - \underline{h}_1(v)\right] dv + \int_p^u \left[\overline{h}_2(v) - \underline{h}_2(v)\right] dv, \tag{4.2}$$

where the slope and intercept for $\overline{h}$ and $\underline{h}$ are decided by l, u, and p using the same procedure. For example, $\overline{\alpha}_1$ is computed by $(\sigma(p) - \sigma(l))/(p - l)$ and $\overline{\beta}_1$ is determined accordingly as $\sigma(l) - \overline{\alpha}_1\, l$. The optimal split point is then $p^* = \arg\min_p P(\sigma, l, u, p)$.

This construction never loosens the local abstraction. The integrated over-approximation

gap is subadditive under a partition of the interval: for any interior point p, each affine envelope built on a sub-interval is at least as tight as the global envelope restricted to that sub-interval, so $\mathcal{L}(\sigma, l, p) + \mathcal{L}(\sigma, p, u) \leq \mathcal{L}(\sigma, l, u)$, with equality only when σ is already affine on $[l, u]$. The unsplit abstraction corresponds to splitting at an endpoint ($p = l$ or $p = u$), where one sub-interval has zero width and contributes no area while the other reproduces the full-interval loss; since the enumeration includes both endpoints, the no-split option lies in the feasible set over which p^* is minimized. The relaxation on each branch is therefore guaranteed to be no looser than the parent interval, and the search falls back to leaving the interval unsplit exactly when no point yields a strictly smaller gap.

Because p^* depends only on the activation function σ and the bound pair (l, u), we pre-compute it offline over a discretized grid and store the results in a lookup table for efficient retrieval at verification time.

We discretize the pre-activation range $[-5,\, 5]$ into a uniform grid $\mathcal{G} = \{\, g_k \mid g_k = -5 + k\delta,\ k = 0, \ldots, N-1 \,\}$ with step size $\delta = 0.01$ and $N = 1001$ grid points. For every ordered index pair $(i,\, j)$ with $0 \leq i < j \leq N-1$, we treat g_i and g_j as the lower and upper pre-activation bounds, enumerate all grid points $p \in \mathcal{G} \cap [g_i,\, g_j]$ as split-point candidates with endpoints included and select the one minimizing the combined tightness loss defined in Equation (4.2). A zero-width segment carries zero relaxation loss, so splitting at an endpoint ($p = g_i$ or $p = g_j$) collapses one side to zero area and reproduces the full-interval loss; the endpoints therefore encode the "do not split" option, which wins exactly when no interior point yields a strictly smaller gap.

For a general sub-interval $[a,\, b] \in \mathcal{G}$, the affine upper bound line $\overline{h}(x) = \overline{\alpha}\, x + \overline{\beta}$ and lower bound line $\underline{h}(x) = \underline{\alpha}\, x + \underline{\beta}$ are obtained by the same linear relaxation procedure as in Equation (3.1), with slopes and intercepts determined by the endpoints a and b. The segment tightness loss then reduces in closed form to:

$$\mathcal{L}(\sigma,\, a,\, b) \;=\; \frac{\overline{\alpha} - \underline{\alpha}}{2}\big(b^2 - a^2\big) + \big(\overline{\beta} - \underline{\beta}\big)(b - a). \tag{4.3}$$

The lookup table entry for each valid pair $(g_i,\ g_j)$ stores the optimal split point:

$$T[i,\ j]\ =\ \underset{p\ \in\ \mathcal{G}\cap[g_i,\,g_j]}{\arg\min}\Big[\ \mathcal{L}(\sigma,\ g_i,\ p)+\mathcal{L}(\sigma,\ p,\ g_j)\ \Big] \tag{4.4}$$

Algorithm 2 BUILDLOOKUPTABLE: Offline Pre-computation of p^*

Input: Activation σ, grid range $B = 5$, step size $\delta = 0.01$
Output: Lookup table $T \in \mathbb{R}^{N\times N}$

$N \leftarrow \lfloor 2B/\delta \rfloor + 1$
$\mathcal{G} \leftarrow \{g_0, g_1, \ldots, g_{N-1}\}$, where $g_k = -B + k\delta$ for $k = 0, 1, \ldots, N-1$
$\mathcal{L}(\sigma,\ g_k,\ g_k) \leftarrow 0$ for all k ▷ zero-width segment: no-split option
Initialize $T[i, j] \leftarrow \mathrm{NaN}$ for all $i,\ j$
for all index pairs $(i,\ j)$ with $0 \le i < j \le N-1$ **do**
    for each candidate index $k \in \{i, i+1, \ldots, j\}$ **do**
        $\mathrm{cost}[k] \leftarrow \mathcal{L}(\sigma,\ g_i,\ g_k) + \mathcal{L}(\sigma,\ g_k,\ g_j)$
    end for
    $T[i, j] \leftarrow g_{\arg\min_k\ \mathrm{cost}[k]}$
end for
return T

At verification time, given actual pre-activation bounds $(l,\ u)$, we locate the nearest grid indices $i_l = \arg\min_s\ |g_s - l|$ and $j_u = \arg\min_s\ |g_s - u|$, and retrieve $T[i_l,\ j_u]$ as the candidate p^*. Because the enumeration includes both endpoints, p^* may coincide with l or u, which signals that no split tightens the relaxation and the neuron is left unrefined. We clamp p^* to $[l,\ u]$ so that such an endpoint optimum survives grid rounding at the query boundaries, and fall back to the midpoint $(l+u)/2$ only in the degenerate case where the stored entry is NaN (i.e. $j_u = i_l$). Algorithm 3 formalizes this procedure.

Algorithm 3 QUERYLOOKUPTABLE: Online p^* Retrieval

Input: Lookup table T, grid $\mathcal{G} = \{g_0, \ldots, g_{N-1}\}$, pre-activation bounds $(l,\ u)$
Output: Split point p^*

$i_l \leftarrow \arg\min_s\ |g_s - l|, \quad j_u \leftarrow \arg\min_s\ |g_s - u|$
$p^* \leftarrow T[i_l,\ j_u]$
if $p^* = \mathrm{NaN}$ **then**
    return $(l+u)/2$
end if
return $\min(\max(p^*,\ l),\ u)$

To illustrate how the enumeration selects p^*, consider RELU on a cross-zero interval

$l = -2$, $u = 3$, where the linear relaxation on $[a,\, b]$ with $a < 0 < b$ gives

$$\overline{\alpha} = \underline{\alpha} = \frac{b}{b-a}, \quad \overline{\beta} = \frac{-ab}{b-a}, \quad \underline{\beta} = 0.$$

Substituting these into Equation (4.3), the first term vanishes since $\overline{\alpha} = \underline{\alpha}$, which leaves $\mathcal{L}(\text{ReLU},\, a,\, b) = -ab$, which gives $\mathcal{L}(\text{ReLU},\, -2,\, 3) = 6$ for the unsplit interval. At $p = 0$, the left branch $[-2,\, 0]$ reduces ReLU to the zero function and the right branch $[0,\, 3]$ reduces it to the identity; both are exact with $\mathcal{L} = 0$, yielding a total loss of 0. At $p = 1$, the left branch $[-2,\, 1]$ remains cross-zero with $\mathcal{L}(\text{ReLU},\, -2,\, 1) = 2$, and the right branch $[1,\, 3]$ is all-positive and exact, giving a total loss of 2. Since $0 < 2$, the enumeration selects $p^* = 0$, confirming that splitting at zero uniquely achieves zero tightness loss and eliminates the triangular over-approximation entirely.

When the interval is same-sign, no split yields any improvement. Consider $l = 1$, $u = 3$ (all-positive): ReLU coincides with the identity on $[1,\, 3]$, so $\overline{\alpha} = \underline{\alpha} = 1$ and $\overline{\beta} = \underline{\beta} = 0$, and the closed-form loss in Equation (4.3) evaluates to zero for the full interval. Since no candidate p can reduce the loss below zero, the enumeration finds no beneficial split. The same argument holds for all-negative intervals such as $l = -3$, $u = -1$, where ReLU is identically zero and the relaxation is again exact. The same mechanism applies to smooth activations such as Tanh and Sigmoid, where the enumeration identifies a p^* that need not coincide with zero or the midpoint, but is instead determined by the curvature of the activation over each sub-interval.

In the LSTM setting, the same p^* rule is applied to the one-dimensional pre-activation interval of each gate ($\mathbf{y}^{i(t)}, \mathbf{y}^{f(t)}, \mathbf{y}^{g(t)}, \mathbf{y}^{o(t)}$) rather than to the two-dimensional gate-cell product surface of Section 3.4. For a selected gate neuron we query the lookup table for its activation (Tanh or Sigmoid) to obtain the optimal split point p^* and partition that gate interval into two branches; on each branch the narrowed gate interval is then passed back through the affine-plane construction of Section 3.4 to recompute the bounds on the gate-cell products. The role of p^* in an LSTM is therefore to tighten the one-dimensional gate relaxations first, so that the subsequent two-dimensional planar relaxation over the gate-cell products is itself constructed over a smaller domain and is correspondingly tighter.

Algorithm 4 describes the branch generation and bound recomputation procedure, and Algorithm 5 describes the depth-limited recursive splitting. In both, $\mathcal{H}_{ref}$ records the interval refinements accumulated along the current path from prior splits, and each branch produces an updated state ($\mathcal{H}_{pos}$ or $\mathcal{H}_{neg}$) that augments $\mathcal{H}_{ref}$ with the new bound on neuron (t_c, r^*).

Algorithm 4 SPLIT: Branch Generation and Bound Computation

Input: Clean input sequence $\mathbf{X}$, perturbation radius ε, norm p, critical neuron (t_c, r^*), refined state $\mathcal{H}_{ref}$
Output: Branch bounds γ_{pos}, γ_{neg} and updated refined states $\mathcal{H}_{pos}$, $\mathcal{H}_{neg}$

Apply $\mathcal{H}_{ref}$ to current $[\mathbf{l}, \mathbf{u}]$ and copy bounds to the positive and negative branches
Determine the split point p^* for (t_c, r^*) by the rule of Section 4.2
Set $l^{(t_c)}_{\text{pos},r^*} \leftarrow p^*$ and $u^{(t_c)}_{\text{neg},r^*} \leftarrow p^*$
Re-run FBC for $k = t_c + 1, \ldots, m$ in each branch
$\gamma_{pos} \leftarrow \text{BBR}(\text{positive branch}), \quad \gamma_{neg} \leftarrow \text{BBR}(\text{negative branch})$
return $(\gamma_{pos},\ \gamma_{neg},\ \mathcal{H}_{pos},\ \mathcal{H}_{neg})$

Algorithm 5 DIVIDE: Depth-Limited Recursive Splitting

Input: Clean input sequence $\mathbf{X}$, perturbation radius ε, norm p, ground-truth label j, current depth d, maximal depth d_{max}, selected neurons $\mathcal{N}_{sel}$, refined state $\mathcal{H}_{ref}$
Output: True iff all induced branches can be certified within the depth budget

if $d \geq d_{\text{max}}$ **or** $d \geq |\mathcal{N}_{sel}|$ **then**
    return False
end if
$(t_c, r^*) \leftarrow \mathcal{N}_{sel}[d]$ (the neuron at position d, indexed from 0)
$(\gamma_{pos}, \gamma_{neg}, \mathcal{H}_{pos}, \mathcal{H}_{neg}) \leftarrow \text{SPLIT}(\mathbf{X}, \varepsilon, p, (t_c, r^*), \mathcal{H}_{ref})$
$V_{pos} \leftarrow \gamma^L_{pos,j} > \max_{i \neq j} \gamma^U_{pos,i}$
$V_{neg} \leftarrow \gamma^L_{neg,j} > \max_{i \neq j} \gamma^U_{neg,i}$
if V_{pos} **and** V_{neg} **then**
    return True
end if
if not V_{pos} **then**
    $V_{pos} \leftarrow \text{DIVIDE}(\mathbf{X}, \varepsilon, p, j, d{+}1, d_{\text{max}}, \mathcal{N}_{sel}, \mathcal{H}_{pos})$
end if
if not V_{neg} **then**
    $V_{neg} \leftarrow \text{DIVIDE}(\mathbf{X}, \varepsilon, p, j, d{+}1, d_{\text{max}}, \mathcal{N}_{sel}, \mathcal{H}_{neg})$
end if
return V_{pos} **and** V_{neg}

4.3 SHAP-Guided Neuron Selection

Splitting every candidate neuron would produce a search tree that grows exponentially with the number of candidates, making exhaustive refinement intractable. Instead, we use gradient-based SHAP attributions to identify the neurons whose relaxation error contributes most to the verification objective, and restrict refinement to that high-impact subset.

Concretely, let $\Phi^{(t)} \in \mathbb{R}^s$ denote the neuron-wise Grad-SHAP attributions for the ground-truth logit F_j with respect to the hidden vector $\mathbf{a}^{(t)}$ at timestep t, where s is the hidden size. For each neuron (t, r), we use the attribution magnitude $|\Phi_r^{(t)}|$ as a proxy for the potential bound improvement from splitting that neuron. The top ρ candidates by this score form the refinement list $\mathcal{N}_{sel}$, which is then sorted in ascending order of t.

The temporal ordering is a deliberate design choice rather than a tie-breaking heuristic. A split at timestep t_c modifies the feasible hidden-state set at that step, and the tighter bounds carry forward through the recurrent transition for all subsequent steps $t_c{+}1, \ldots, m$. Processing earlier timesteps first therefore amplifies the benefit of each split before later candidates are evaluated, maximizing bound improvement per branch under a fixed depth budget.

Because SELECT admits all neurons and ranks them purely by attribution magnitude, the same procedure serves both RELU and smooth activations. For smooth activations every neuron carries nonzero relaxation error, so any selected neuron benefits from refinement; for RELU, only sign-changing neurons carry error, and splitting a single-sign neuron simply leaves the bounds unchanged. Algorithm 6 describes the full selection procedure.

Algorithm 6 SELECT: Critical Neuron Selection

Input: Clean input sequence $\mathbf{X}$, perturbation radius ε, norm p, ground-truth label j, selection count ρ

Output: $\mathcal{N}_{sel}$: neurons $\{(t, r)\}$ sorted ascending by t

Sample background $\mathcal{B} \subset \mathbb{B}_p(\mathbf{X}, \varepsilon)$
$m \leftarrow$ sequence length of $\mathbf{X}$
for $t = 1$ to m **do**
    $\Phi^{(t)} \leftarrow$ Grad-SHAP attributions of F_j with respect to $\mathbf{a}^{(t)}$ over $\mathcal{B}$
    for each neuron r **do**
        Add $(t,\ r,\ |\Phi_r^{(t)}|)$ to candidate list $\mathcal{C}$
    end for
end for
$K \leftarrow \min(\rho,\ |\mathcal{C}|)$
return top-K neurons (t, r) from $\mathcal{C}$ ranked by $|\Phi_r^{(t)}|$, sorted ascending by t

4.4 Illustrative Example

Continuing from the running example in Section 3.3, suppose SHAP ranks neuron $(1, 2)$ ahead of $(2, 4)$ because $|\Phi_2^{(1)}| > |\Phi_4^{(2)}|$. The selection is consistent with the temporal-ordering intuition: any relaxation error introduced at timestep 1 is carried through the recurrent transition before the output is computed, making early neurons disproportionately harmful.

We therefore split neuron $(1, 2)$ first, producing a positive branch with $l_{\text{pos},2}^{(1)} \leftarrow 0$ and a negative branch with $u_{\text{neg},2}^{(1)} \leftarrow 0$. After re-running FBC and BBR under the two refined branches, suppose the positive branch yields $\gamma_j^L = 0.23$ and $\max_{i \neq j} \gamma_i^U = 0.19$, while the negative branch yields $\gamma_j^L = 0.27$ and $\max_{i \neq j} \gamma_i^U = 0.17$. Both branches now satisfy Equation (4.1), so the original sample is certified without ever splitting neuron $(2, 4)$. This illustrates the core benefit of SHAP-guided single-neuron refinement: by removing the dominant relaxation error first and allowing the backward recursion to tighten the remaining bounds, we avoid exhaustive branching over all candidates.

5 Experiment Evaluation

We evaluate our method on two RNN settings and one LSTM setting, addressing four research questions: (RQ1) certification gains over abstraction-only verification, (RQ2) runtime overhead of abstraction refinement across activation types and sequence lengths, (RQ3) computational cost of SHAP-guided neuron selection, and (RQ4) effectiveness of abstraction refinement when extended to LSTMs.

5.1 Experimental Setup

5.1.1 Dataset and Models

We train RNNs of hidden layer sizes $h \in \{16, 32, 64, 128\}$ on two datasets: CIFAR10 and MNIST Stroke sequences (Liwicki et al., 2012). CIFAR10 images are reshaped into sequences with $m \in \{8, 12, 24, 32\}$, where each frame corresponds to a horizontal slice of pixels. MNIST Stroke sequences record pen trajectory coordinates and have sequence lengths $m \in \{30, 35, 40, 45\}$. Additionally, we train LSTM classifiers of hidden size $h \in \{4, 8, 16, 32\}$ on MNIST images with sequence lengths $m \in \{1, 2, 4, 7\}$.

5.1.2 Property and Protocol

We certify top-1 label preservation under per-timestep L_2 perturbations: for a clean input $\mathbf{X}_0$ with ground-truth label $j = \arg\max_i F_i(\mathbf{X}_0)$, we prove $F_j(\mathbf{X}) > F_i(\mathbf{X})$ for all $i \neq j$ and all $\mathbf{X}$ in the ε-ball at each timestep. We use the first 50 samples from each dataset and apply refinement without filtering based on prediction correctness.

5.1.3 Evaluation Metrics

We adopt an EVR-style ε sweep following prior work (Li et al., 2023). For each sample, ε is swept linearly from 0.005 to 0.1 in increments of 0.001. As long as the original abstraction certifies robustness, the sweep continues without refinement. When the baseline first fails, the refined abstraction is invoked. If the refined abstraction certifies the sample at that ε, the sample is counted as a recovery ($\uparrow$); otherwise, the sweep continues. In the result tables, $\times$ denotes the number of samples for which refinement is triggered at some point during the sweep, $\uparrow$ denotes the subset recovered by refinement, and the refinement rate (%) is the ratio $\uparrow / \times$, that is, the fraction of triggered samples that refinement successfully recovers. We use the refinement rate as the primary effectiveness metric throughout this chapter.

5.1.4 SHAP-Guided Neuron Selection

We ranked neurons using a gradient-based explainer (Sundararajan et al., 2017) computed on hidden states with randomly perturbed backgrounds, then restrict candidates to neurons whose pre-activation interval straddles zero. We keep the top five candidates as the split list. At each recursive split, we choose the first candidate that is temporally valid, has not been split before, and still crosses zero under the refined pre-activation bounds. For LSTMs, we do not enforce the zero-crossing constraint and instead rank all neurons by importance. We allow at most 31 refinements per configuration.

5.1.5 Implementation Details

All models and verification scripts are implemented in PyTorch on top of POPQORN's bound propagation framework.

5.2 Certification Success Rate and Model Sensitivity (RQ1)

Tables 5.1 and 5.2 show that abstraction refinement improves certification in nearly all settings. In particular, it recovers samples that are in fact robust but are reported as unsafe

Table 5.1: EVR improvements on CIFAR10

m	Act.	$h = 16$			$h = 32$			$h = 64$			$h = 128$		
		$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%
8	RELU	43	24	56	47	33	70	50	32	64	50	25	50
	TANH	50	35	70	50	28	56	50	30	60	50	25	50
12	RELU	47	16	34	50	20	40	50	26	52	50	22	44
	TANH	50	34	68	50	27	54	50	15	30	50	11	22
24	RELU	50	16	32	50	19	38	50	12	24	50	8	16
	TANH	50	21	42	50	13	26	50	11	22	50	1	2
32	RELU	50	18	36	50	17	34	50	12	24	50	4	8
	TANH	50	16	32	50	10	20	50	4	8	50	1	2

Table 5.2: EVR improvements on MNIST Strokes.

m	Act.	$h = 16$			$h = 32$			$h = 64$			$h = 128$		
		$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%
30	RELU	50	20	40	50	24	48	50	10	20	50	4	8
	TANH	50	30	60	50	19	38	50	9	18	50	3	6
35	RELU	50	15	30	50	17	34	50	6	12	50	4	8
	TANH	50	21	42	50	11	22	50	4	8	50	0	0
40	RELU	50	13	26	50	17	34	50	7	14	50	3	6
	TANH	50	22	44	50	2	4	50	2	4	50	0	0
45	RELU	50	5	10	50	11	22	50	3	6	50	6	12
	TANH	50	16	32	50	9	18	50	6	12	50	1	2

Note: $\times$ is the number of samples triggering refinement, $\uparrow$ the number recovered, and % the refinement rate $\uparrow / \times$.

due to over-approximation errors in the abstraction, thereby reducing conservativeness.

A consistent pattern observed in both CIFAR10 and MNIST Stroke is that the refinement rate gradually diminishes with increasing m. The accumulation of relaxation error with increasing m amplifies over-approximation effects, so that under a fixed splitting budget, recovering the samples that the abstraction alone previously missed becomes harder. The effect is significant: on CIFAR10 at $h = 128$, the RELU refinement rate falls from 50% at $m = 8$ to 8% at $m = 32$, and TANH falls from 50% to 2% over the same range; on MNIST Stroke at $h = 64$, RELU drops from 20% at $m = 30$ to 6% at $m = 45$. This monotonic decline across both datasets indicates that temporal depth is the key bottleneck for refinement.

Model width exhibits a similar but less consistent trend. Overall, wider models are more difficult to recover under the same split budget: as width increases, the pool of can-

didate neurons grows, and under a fixed selection count and budget, locating the few splits that meaningfully tighten the bound becomes harder. The primary exception is RELU, which tends to peak at intermediate hidden sizes rather than declining monotonically. For instance, on CIFAR10 at $m = 8$ the RELU refinement rate rises from 56% at $h = 16$ to 70% at $h = 32$ before falling to 64% at $h = 64$ and 50% at $h = 128$, so the peak sits at $h = 32$ rather than at the smallest width. Because splitting a selected RELU neuron makes its abstraction exact, it eliminates the dominant source of relaxation error, and this effect is largely independent of how many other candidate neurons remain.

The impact of activation functions varies with model width. Comparing refinement rates, TANH outperforms RELU when the hidden size is small: on CIFAR10 at $h = 16$, the TANH refinement rate exceeds RELU at $m = 8$ (70% versus 56%) and at $m = 12$ (68% versus 34%), and the same ordering holds on MNIST Stroke at $h = 16$ for $m = 30$ (60% versus 40%). However, this ordering reverses as the width h increases, with RELU benefiting more from scaling. This can be explained by the accumulation of relaxation error in wider models. As the number of neurons grows, the overall approximation error becomes more significant. Our refinement mitigates this effect for RELU by splitting on selected neurons, which renders their relaxation exact. As the backward recursion proceeds, splits applied at earlier timesteps further tighten the linear relaxations at subsequent timesteps, amplifying the overall certification. In contrast, since the relaxation error of TANH cannot be fully eliminated in the same manner, it continues to accumulate with model width, and therefore does not benefit as much as RELU even within the restricted sub-interval.

5.3 Runtime Overhead vs. Abstraction Baseline (RQ2)

Figures 5.1 and 5.2 compare the runtime of the refinement against the abstraction-only baseline. Figure 5.1 illustrates the average verification time across the complete ε sweep for each configuration, while Figure 5.2 presents the runtime ratio of refinement to the baseline, averaged over different hidden sizes.

The verification time for both methods increases significantly as m grows. This is primarily because longer sequence unrollings inherently require more bound computa-

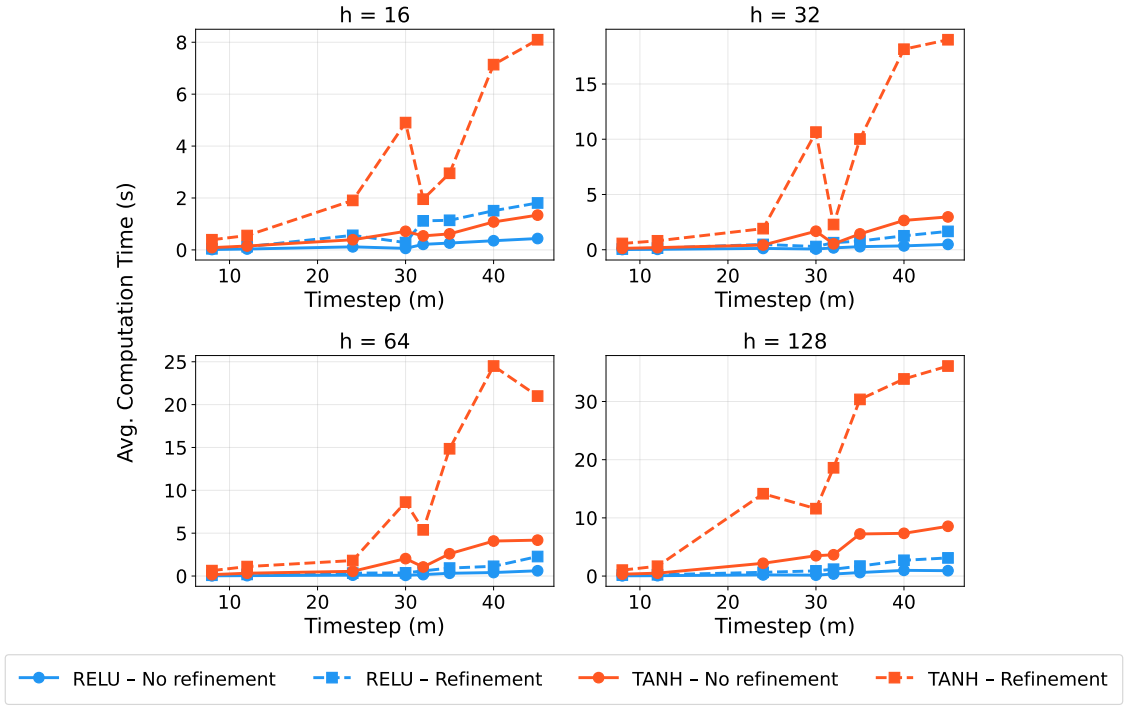


Figure 5.1: Average computation time of the baseline and refinement across m under different h.

tion steps. Furthermore, because refinement introduces the expected computational overhead of branch generation and bound recomputation, it consistently runs slower than the baseline across all tested conditions. Although the refinement-to-baseline runtime ratio fluctuates with m, it remains strictly greater than 1. This indicates that refinement fundamentally offers a predictable trade-off between accuracy and runtime, rather than yielding any end-to-end speedup.

Within each subplot of Figure 5.1, computation time also rises steadily as h increases. The underlying reason is that the cost of bound computation at each timestep is dominated by the per-neuron envelope construction: each hidden neuron requires its own affine relaxation, so widening the hidden layer linearly increases the per-step abstraction work, which accumulates over the unrolled sequence.

Regarding the activation functions, the charts reveal a stable and informative disparity: TANH is consistently and noticeably more computationally expensive than RELU. This occurs because, for TANH, each refined branch still necessitates the recomputation of smooth activation envelopes within the restricted sub-interval. Conversely, splitting a RELU neuron makes its state exact, particularly on the negative region which becomes 0, thereby

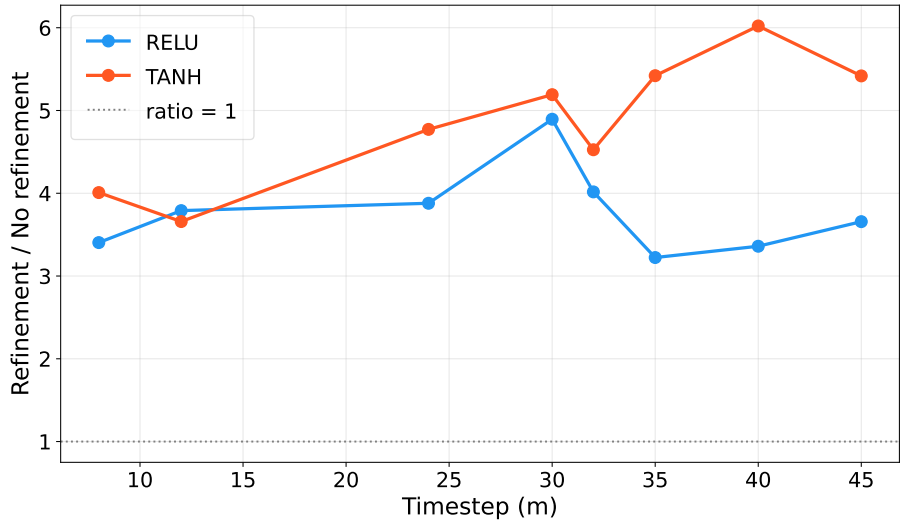


Figure 5.2: Computation time ratio of refinement vs. no refinement.

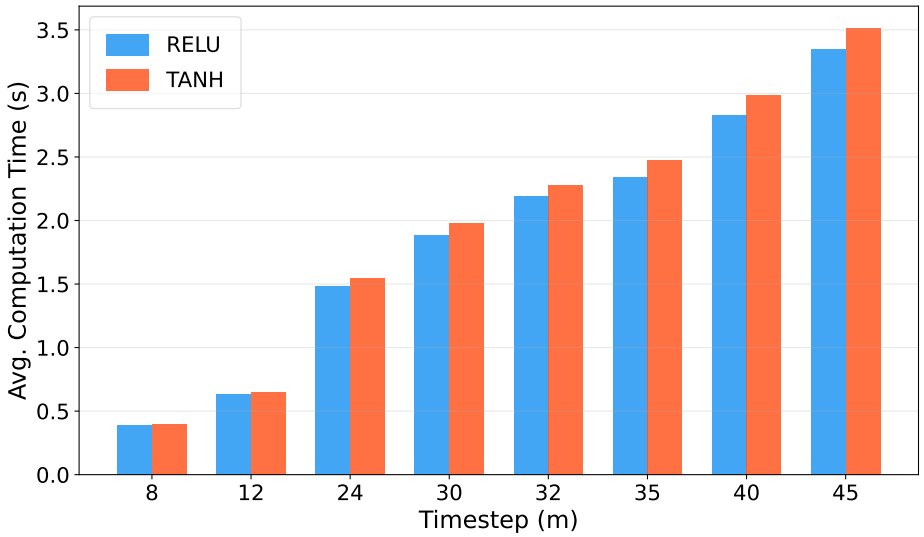


Figure 5.3: Computation time for SHAP scores across sequence lengths.

eliminating the intermediate linear abstraction time for these negative-interval branches. This timing gap, driven by the distinct properties of the activation functions, is evident across all sequence lengths and hidden sizes, and the disparity widens as m increases. This growing gap is due to the fact that each additional branch requires a complete recomputation pass.

Table 5.3: EVR improvements on Vanilla RNN over MNIST

m	Act.	$h=4$			$h=8$			$h=16$			$h=32$		
		×	↑	%	×	↑	%	×	↑	%	×	↑	%
1	RELU	9	1	11	2	0	0	0	0	-	0	0	-
	TANH	7	0	0	3	0	0	0	0	-	2	2	100
2	RELU	13	2	15	5	1	20	3	2	67	4	1	25
	TANH	22	7	32	12	4	33	5	5	100	2	2	100
4	RELU	30	12	40	9	4	44	4	3	75	1	0	0
	TANH	22	7	32	24	20	83	17	17	100	8	8	100
7	RELU	43	8	19	25	16	64	10	9	90	4	2	50
	TANH	45	24	53	35	30	86	27	23	85	41	41	100

5.4 Efficiency vs. Refinement Effectiveness (RQ3)

Figure 5.3 demonstrates that SHAP computation time increases steadily with sequence length, with RELU and TANH showing consistently similar timings across all timesteps. The negligible gap between activation types confirms that candidate scoring overhead is governed primarily by the length of the unrolled hidden-state sequence over which gradient attributions must be computed, not by activation-specific characteristics.

Unlike branch verification, this cost is incurred only once per sample and subsequently amortized across all refinement calls within the same ε sweep. SHAP thus serves as a lightweight, one-time filtering step that narrows a potentially large candidate set down to a compact, high-priority refinement list. Combined with the verification gains in Section 5.2 and the runtime analysis in Section 5.3, these results confirm that branch verification constitutes the dominant computational bottleneck, while SHAP's targeted candidate selection effectively concentrates that budget on the neurons most likely to yield a tighter robustness certificate.

5.5 Effectiveness of Abstraction Refinement on LSTMs (RQ4)

We evaluate our refinement strategy on small LSTM models to assess whether it extends effectively to gated architectures. Table 5.4 and Table 5.3 show the comparison under

Table 5.4: EVR improvements on LSTM over MNIST

m	Act.	$h = 4$			$h = 8$			$h = 16$			$h = 32$		
		×	↑	%	×	↑	%	×	↑	%	×	↑	%
1	SIGMOID/TANH	44	11	25	40	22	55	27	23	85	16	14	88
2	SIGMOID/TANH	50	19	38	49	38	78	43	32	74	34	23	68
4	SIGMOID/TANH	50	24	48	50	23	46	50	24	48	50	18	36
7	SIGMOID/TANH	50	11	22	50	15	30	50	8	16	50	8	16

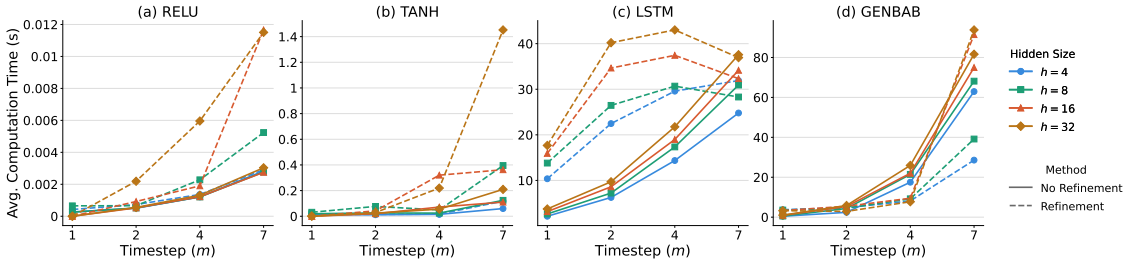


Figure 5.4: Average computation time of vanilla RNNs, LSTMs, and the GenBaB baseline on MNIST across m under different h. Subplots (a) RELU and (b) TANH are vanilla RNNs, (c) is our LSTM refinement, and (d) is the GenBaB baseline. Solid lines denote abstraction only; dashed lines denote refinement.

the same configuration of timesteps and hidden sizes on MNIST.

The number of recovered samples in LSTMs follows a rise-then-fall pattern: it climbs from $m = 1$ to a peak at short-to-intermediate lengths–$m = 2$ for $h \in \{8, 16, 32\}$ and $m = 4$ for $h = 4$–before dropping to its lowest at $m = 7$. The rise reflects a growing candidate pool: as m increases, more samples fail the baseline and enter refinement, and at $m = 2$ a single split can still tighten most of the accumulated relaxation, giving the greatest recovery. Beyond that point the pool saturates ($\times = 50$ across all widths), but each additional timestep stacks four more gate relaxations, so one split removes a steadily smaller share of the total error and recovery declines. Vanilla RNNs over the same MNIST range exhibit the opposite trend, with recovery growing as m increases, because longer sequences expose more productive cross-zero candidates while each timestep introduces only a single hidden-state relaxation rather than four coupled gate relaxations. The two architectures therefore diverge increasingly with m: at $m = 1$ the vanilla RNN abstraction already certifies most samples, leaving little room for refinement, whereas baseline failures in LSTMs remain prevalent even at $m = 1$ due to the compounded gate relaxations; and at $m = 7$ the per-split gain in LSTMs collapses under four-gate stacking while vanilla RNNs continue to benefit. Combined with the results in Section 5.2, this confirms that sequence

length is a primary driver of the verification gain in both architectures, though its effect manifests earlier and more severely in LSTMs.

Both architectures share a consistent pattern across model width: the refinement rate rises at intermediate hidden sizes before declining at larger widths. At $h = 4$, the candidate pool is too small for SHAP-guided selection to reliably locate a neuron whose split meaningfully tightens the reachable set, as splitting at zero in vanilla RNNs or at p^* in LSTMs requires a neuron that individually carries sufficient influence over the bound. As width grows to moderate sizes, the richer pool allows SHAP to identify impactful candidates, and the refinement rate peaks around $h = 8$ for LSTMs and $h = 16$ for vanilla RNNs. The earlier peak in LSTMs follows from their four-gate structure: the verifier must propagate bounds through four times as many nonlinear operations per timestep, so a single split recaptures a smaller fraction of the total relaxation error as h grows. Beyond each peak, the same effect generalizes: the per-split gain shrinks as more neurons collectively determine the bound, and wider models also yield tighter baseline abstractions, leaving fewer but harder cases for refinement to recover.

Figure 5.4 reports the average per-refinement computation time of vanilla RNNs and LSTMs on MNIST across m under different h. LSTM refinement is costlier by two to three orders of magnitude than vanilla RNNs, as each split requires recomputing bounds through four gate transformations per timestep, each demanding an independent hyperplane approximation over SIGMOID and TANH activations. Computation time grows monotonically with h, since wider gates directly increase the cost of each recomputation. Along m, the growth diminishes at larger sequence lengths: heavy relaxation errors cause most splits to exhaust the refinement budget without recovery, terminating each ε early rather than completing a full recomputation pass, which partially offsets the cost of longer unrolling.

5.6 Comparison with the GenBaB Baseline

To situate our method within the broader landscape of refinement-based verifiers, we compare against GenBaB (Shi et al., 2025), a recent branch-and-bound verifier for general nonlinearities, under the identical MNIST LSTM configuration ($h \in \{4, 8, 16, 32\}$, $m \in$

Table 5.5: Benchmark results of GenBaB on LSTM over MNIST

m	Act.	$h=4$			$h=8$			$h=16$			$h=32$		
		$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%	$\times$	$\uparrow$	%
1	SIGMOID/TANH	10	10	100	11	11	100	8	8	100	3	3	100
2	SIGMOID/TANH	15	15	100	12	12	100	5	5	100	4	4	100
4	SIGMOID/TANH	11	11	100	8	8	100	7	7	100	6	6	100
7	SIGMOID/TANH	16	16	100	11	11	100	49	49	100	49	49	100

$\{1, 2, 4, 7\}$), the same first 50 samples, the same per-timestep L_2 property, and the same ε sweep. Results are reported in Table 5.5. The comparison is not like-for-like, since the two systems refine different baselines: our pipeline refines POPQORN, whereas GenBaB is built on α, β-CROWN (Wang et al., 2021, Xu et al., 2021). Because the metric $\times$ counts samples for which the baseline fails during the sweep, a tighter baseline yields a smaller $\times$, so the $\uparrow / \times$ ratios of the two systems measure recovery over residual sets of different difficulty. We read the two tables with this in mind.

GenBaB triggers refinement on fewer samples than our method—for example $\times = 10$ at $m = 1, h = 4$ against $\times = 44$ for ours—and recovers all of them (100% across every configuration). This is consistent with the tighter α, β-CROWN baseline, which certifies more samples before any branching and therefore leaves a smaller and easier residual set. The contrast is sharpest in the long-sequence, wide-model regime: at $m = 7$ with $h \in \{16, 32\}$ GenBaB triggers on $\times = 49$ samples and still recovers all of them, whereas our method recovers 16% of its triggered samples in the same setting. GenBaB's branch-and-bound machinery thus holds a clear advantage on the hardest long-sequence cases.

The aim of this comparison is to position our method, not to rank it above GenBaB. Our contribution is a lightweight refinement layer over POPQORN, and its three components—SHAP-guided temporal ordering, single-neuron refinement, and the p^* lookup table—are independent of the underlying verifier; Chapter 6 discusses how they could be layered onto a tighter baseline such as α, β-CROWN.

The runtime behavior of the two systems also differs structurally, as Figure 5.4(d) shows. GenBaB inherits the cold start of α, β-CROWN: its initial abstraction builds the full computation graph, derives pre-activation bounds for every neuron, and runs gradient ascent to optimize the relaxation of each neuron toward the tightest output bounds,

which dominates its cost. At refinement time these intermediate quantities are already available, so a split does not trigger a full recomputation; only the bounds from the split timestep onward are updated. Consequently, GenBaB's refinement adds little over its own abstraction, and for the configurations where few samples trigger refinement ($h \in \{4, 8\}$ at $m = 7$) the refinement-enabled runtime even falls below the abstraction-only runtime as m grows. This contrasts with our POPQORN-based pipeline (Figure 5.4(a)–(c)), in which each branch re-runs FBC and BBR over all timesteps $k = t_c + 1, \ldots, m$ and is therefore strictly slower than the baseline in every configuration.

6 Conclusions

This thesis propose a targeted abstraction-refinement framework for certified local robustness verification of RNNs. When abstraction-only bound computation is too conservative to certify a sample, we split the pre-activation interval of a small number of SHAP-identified critical neurons, recompute the bounds through the remaining unrolled timesteps, and certify each branch independently. For RELU the split is placed at zero, which makes the selected neuron exact, and for the smooth gate pre-activation of LSTMs it is placed at a precomputed point p^* that minimizes the integrated relaxation gap. In both cases the refinement is applied in temporal order, so that a split at an early timestep tightens the bounds at every subsequent recurrent transition. The framework thus combines a principled splitting rule, an attribution-guided selection of high-impact neurons, and a recurrence-aware ordering that concentrates a limited budget where it matters most.

Our evaluation across two vanilla RNN benchmarks and one LSTM benchmark shows that the refinement consistently recovers samples that abstraction alone reports as unsafe, while incurring a predictable accuracy-for-runtime trade-off. The dominant factor governing both the certification gain and its limits is sequence length: as the recurrence is unrolled over more timesteps, relaxation error accumulates and a fixed split budget recovers a shrinking fraction of failing samples, an effect that appears in both vanilla RNNs and, more sharply, in LSTMs, where four gate computations per timestep introduce error along multiple pathways that a single split cannot fully tighten. Model width plays a weaker role, and the SHAP-based selection adds only a one-time cost per sample, leaving branch verification as the principal computational bottleneck.

Taken together, these results substantiate the thesis: that abstraction refinement for RNNs is most effective when it is selective rather than exhaustive, and when it respects

the temporal structure of the recurrence. The contribution is threefold, spanning a single-neuron, temporally ordered refinement workflow, a tightness-loss formulation of the split point with an inexpensive lookup-based rule for smooth activations, and a systematic empirical account of how certification gains and runtime scale with activation type, sequence length, and hidden size.

The approach is sound but incomplete: it can still branch exponentially in the worst case, its effectiveness depends on the refinement budget, and its overhead is more pronounced for smooth activations than for RELU. Natural extensions include adaptive refinement schedules that allocate depth and selection effort per sample, finer-grained selection that combines attribution with an estimate of each neuron's relaxation gap, and a fuller treatment of other gated architectures and richer perturbation models. A more ambitious direction is the extension to Transformers, which have largely displaced RNNs on sequential tasks. The three core ingredients of our framework—SHAP-guided neuron selection, the p^* split point for one-dimensional nonlinearities, and the delegation of post-split bounding to an underlying verifier—are by design independent of any particular architecture, so the portable part of the method is clear. A Transformer's position-wise feed-forward sublayers use one-dimensional activations such as RELU or GeLU, exactly the setting our p^* rule already handles; applying it there would require only replacing the underlying planar relaxation with a tool able to bound a Transformer computation graph, such as a CROWN-style verifier or a multi-norm zonotope verifier, while the "refine-then-delegate" structure is preserved. The genuine obstacle lies in the self-attention sublayer. Its query–key inner product is a bilinear, multivariate operation in which two input-dependent quantities are multiplied, and its softmax is built from exponentiation and normalization; neither is a univariate function. This breaks the premise behind p^*, namely that the split point is a single point on a one-dimensional interval $[l, u]$: refining a d-dimensional nonlinear operation would require splitting a d-dimensional pre-activation region, so a single split produces 2^d sub-problems rather than two, with no uniquely optimal split point determined in advance. Moreover, unlike RELU, whose branches become exact after a split at zero, the curved operations in attention remain approximate even after splitting, so each split yields a limited and diminishing marginal gain; combined with the quadratic-

in-sequence-length cost of attention bound propagation, per-region splitting would scale poorly. The temporal ordering at the heart of our method also does not transfer, since attention couples all positions within a single layer rather than through a sequential recurrence, so a selection principle organized around attention structure and layer depth—rather than temporal position—would be needed. Extending the framework to Transformers is therefore feasible but non-trivial: the SHAP-guided selection and the p^* rule carry over directly to the univariate feed-forward activations, whereas the multivariate nonlinearities of self-attention demand a substantial extension of the splitting mechanism from a one-dimensional point to a multi-dimensional region, together with a redesigned selection criterion. We leave this to future work.

Reference

Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L., and Savarese, S. (2016). Social lstm: Human trajectory prediction in crowded spaces. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 961–971.

Bunel, R., Lu, J., Turkaslan, I., Torr, P. H. S., Kohli, P., and Kumar, M. P. (2020). Branch and bound for piecewise linear neural network verification. *Journal of Machine Learning Research*, 21(42):1–39.

Che, Z., Purushotham, S., Cho, K., Sontag, D., and Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085.

Cherny, D., Herasymenko, V., and Semenov, A. (2018). Adversarial machine learning in credit card fraud detection. In *IEEE International Conference on Data Stream Mining & Processing (DSMP)*, pages 351–355.

Cohen, J., Rosenfeld, E., and Kolter, Z. (2019). Certified adversarial robustness via randomized smoothing. In *International Conference on Machine Learning (ICML)*, pages 1310–1320.

Cousot, P. and Cousot, R. (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 238–252.

De Palma, A., Bunel, R., Desmaison, A., Dvijotham, K., Kohli, P., Torr, P. H., and Kumar, M. P. (2021). Improved branch and bound for neural network verification via lagrangian decomposition. *arXiv preprint arXiv:2104.06718*.

Du, T., Ji, S., Shen, L., Zhang, Y., Li, J., Shi, J., Fang, C., Yin, J., Beyah, R., and Wang, T. (2021). Cert-rnn: Towards certifying the robustness of recurrent neural networks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 516–534.

Du, X., Li, Y., Xie, X., Ma, L., Liu, Y., and Zhao, J. (2020). Marble: Model-based robustness analysis of stateful deep learning systems. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Gehr, T., Mirman, M., Drachsler-Cohen, D., Tsankov, P., Chaudhuri, S., and Vechev, M. (2018). Ai2: Safety and robustness certification of neural networks with abstract interpretation. In *IEEE Symposium on Security and Privacy (SP)*, pages 3–18.

Goodfellow, I. J., Shlens, J., and Szegedy, C. (2015). Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*.

Graves, A., Mohamed, A.-r., and Hinton, G. (2013). Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6645–6649.

Hardy, G. H., Littlewood, J. E., and Pólya, G. (1952). *Inequalities*. Cambridge University Press, Cambridge, 2nd edition.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.

Katz, G., Barrett, C., Dill, D. L., Julian, K., and Kochenderfer, M. J. (2017). Reluplex: An efficient smt solver for verifying deep neural networks. In *Computer Aided Verification (CAV)*, pages 97–117. Springer.

Katz, G., Barrett, C., Dill, D. L., Julian, K., and Kochenderfer, M. J. (2019). The marabou framework for verification and analysis of deep neural networks. In *Computer Aided Verification (CAV)*, pages 443–452. Springer.

Ko, C.-Y., Lyu, Z., Weng, L., Daniel, L., Wong, N., and Lin, D. (2019). Popqorn: Quantifying robustness of recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 3468–3477. PMLR.

Li, J., Bai, G., Pham, L. H., and Sun, J. (2023). Towards an effective and interpretable refinement approach for dnn verification. In *IEEE International Conference on Software Quality, Reliability, and Security (QRS)*, pages 569–580. IEEE.

Li, Y., Wen, C., Juefei-Xu, F., and Feng, C. (2021). Fooling lidar perception via adversarial trajectory perturbation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7898–7907.

Liwicki, M., Graves, A., Bunke, H., and Schmidhuber, J. (2012). Recognition of whiteboard notes: Online, offline and combination. In *Series in Machine Perception and Artificial Intelligence*, volume 75, pages 1–13. World Scientific.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, pages 4765–4774.

Madry, A., Makelov, A., Schmidt, L., Tsipras, D., and Vladu, A. (2017). Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*.

Mohammadinejad, S., Paulsen, B., Deshmukh, J. V., and Wang, C. (2021). Diffrnn: Differential verification of recurrent neural networks. In *International Conference on Formal Modeling and Analysis of Timed Systems (FORMATS)*, pages 117–134.

Nelson, D. M., Pereira, A. C., and de Oliveira, R. A. (2017). Stock market's price movement prediction with lstm neural networks. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1419–1426.

Ryou, W., Chen, J., Balunovic, M., Singh, G., Dan, A., and Vechev, M. (2021). Scalable polyhedral verification of recurrent neural networks. In *International Conference on Computer Aided Verification (CAV)*, pages 225–248. Springer.

Shi, Z., Jin, Q., Kolter, Z., Jana, S., Hsieh, C.-J., and Zhang, H. (2025). Neural network verification with branch-and-bound for general nonlinearities. In *Tools and Algorithms*

for the Construction and Analysis of Systems (TACAS), volume 15696 of *Lecture Notes in Computer Science*, pages 315–335. Springer.

Singh, G., Gehr, T., Mirman, M., Püschel, M., and Vechev, M. (2019). An abstract domain for certifying neural networks. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–30.

Singh, G., Gehr, T., Püschel, M., and Vechev, M. (2018). Fast and precise certification of neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 10802–10813.

Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks.

Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3104–3112.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., and Fergus, R. (2013). Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*.

Tran, H.-D., Choi, S., Yang, X., Yamaguchi, T., Hoxha, B., and Prokhorov, D. (2023). Verification of recurrent neural networks with star reachability. In *Proceedings of the 26th ACM International Conference on Hybrid Systems: Computation and Control (HSCC)*, pages 1–13.

Wang, S., Zhang, H., Xu, K., Lin, X., Jana, S., Hsieh, C.-J., and Kolter, J. Z. (2021). Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 29909–29921.

Xu, K., Zhang, H., Wang, S., Wang, Y., Jana, S., Lin, X., and Hsieh, C.-J. (2021). Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers. In *International Conference on Learning Representations (ICLR)*.

Zhang, H., Weng, T.-W., Chen, P.-Y., Hsieh, C.-J., and Daniel, L. (2018). Efficient neural network robustness certification with general activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4939–4948.

Zhang, Y., Albarghouthi, A., and D'Antoni, L. (2021). Certified robustness to programmable transformations in lstms. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1068–1083.

Zhang, Y., Du, T., Ji, S., Tang, P., and Guo, S. (2023). Rnn-guard: Certified robustness against multi-frame attacks for recurrent neural networks. *arXiv preprint arXiv:2304.07980*.

Zhou, D., Brix, C., Hanasusanto, G. A., and Zhang, H. (2024). Scalable neural network verification with branch-and-bound inferred cutting planes. In *NeurIPS*.